

LAPORAN MINGGU 2 PROJECT 1 PEMROGRAMMAN PARALLEL

SIMULASI PELURUHAN PARTIKEL PARALEL CHAIN DECAY URANIUM-238

oleh:

Ahmad Hanafi Hasan (245090300111001)

Stephen Tangguh Laia (245090300111010)

PROGRAM STUDI: S1 FISIKA



DEPARTEMEN FISIKA

FAKULTAS SAINS, TEKNOLOGI, DAN MATEMATIKA

UNIVERSITAS BRAWIJAYA

MALANG

2026

Kode paralel yang digunakan :

```
"""
Simulasi peluruhan rantai U-238 → Pb-206 menggunakan MPI (mpi4py)

=====

CARA MENJALANKAN:

    mpiexec -n <JUMLAH_PROSES> python decay_mpi.py

Contoh:

    mpiexec -n 4 python decay_mpi.py      # 4 proses
    mpiexec -n 8 python decay_mpi.py      # 8 proses (untuk uji skalabilitas)

=====

PARAMETER YANG BISA DIUBAH:

    - N_EVENTS   : jumlah event Monte Carlo (baris ~36)
    - Jumlah proses diatur lewat argumen -n pada mpiexec (bukan di kode)

CATATAN SKALABILITAS:

    Untuk mengukur speedup, jalankan ulang dengan -n 1, 2, 4, 8, 16, dst.
    lalu bandingkan "Waktu komputasi" yang tercetak oleh rank 0.

=====
"""

import math
import time

import matplotlib
import matplotlib.pyplot as plt
import numpy as np
from mpi4py import MPI

# -----
```

```

# PARAMETER UTAMA ← UBAH DI SINI

# -----

N_EVENTS = 100_000    # ← jumlah event total; coba 10_000 / 100_000 /
                        1_000_000

MASTER_SEED = 12345   # seed global untuk reproduibilitas

# -----

# Unit konversi ke satuan TAHUN

DAY          = 1 / 365.25

MINUTE       = 1 / (365.25 * 24 * 60)

SECOND       = 1 / (365.25 * 24 * 3600)

MICROSECOND  = 1e-6 * SECOND

# Rantai peluruhan U-238 → Pb-206 (nama isotop, waktu paruh dalam tahun)

DECAY_CHAIN = [
    ("U-238", 4.468e9),
    ("Th-234", 24.10 * DAY),
    ("Pa-234m", 1.17 * MINUTE),
    ("U-234", 245_500.0),
    ("Th-230", 75_380.0),
    ("Ra-226", 1_600.0),
    ("Rn-222", 3.8235 * DAY),
    ("Po-218", 3.10 * MINUTE),
    ("Pb-214", 26.8 * MINUTE),
    ("Bi-214", 19.9 * MINUTE),
    ("Po-214", 164.3 * MICROSECOND),
    ("Pb-210", 22.3),
    ("Bi-210", 5.012 * DAY),
    ("Po-210", 138.376 * DAY),
    # Pb-206 stabil → rantai berhenti di sini
]

```

```

# Mean lifetime  $\tau = t_{1/2} / \ln 2$ 
MEAN_LIFETIMES = np.array([t / math.log(2) for _, t in DECAY_CHAIN])

def simulate_local(n_local: int, seed: int) -> np.ndarray:
    """
    Setiap proses MPI memanggil fungsi ini untuk mensimulasikan
    n_local event. Setiap event = satu atom U-238 yang meluruh
    melalui seluruh rantai hingga mencapai Pb-206 stabil.

    Waktu tunggu pada setiap tahap rantai berdistribusi eksponensial
    dengan mean =  $\tau_i$  (mean lifetime isotop ke-i).
    Total waktu peluruhan satu event = jumlah semua waktu tunggu.
    """
    rng = np.random.default_rng(seed)
    # shape: (n_local, jumlah_isotop_dalam_rantai)
    waiting_times = rng.exponential(MEAN_LIFETIMES, size=(n_local,
        len(MEAN_LIFETIMES)))
    return waiting_times.sum(axis=1) # total waktu tiap event

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank() # ID proses ini (0 = master/root)
    size = comm.Get_size() # jumlah proses total ← diatur via -n

    # — Distribusi beban —————
    # Rank 0 menghitung berapa event yang dikerjakan masing-masing rank.
    base = N_EVENTS // size
    extra = N_EVENTS % size
    # Rank 0..extra-1 masing-masing mengerjakan satu event lebih banyak
    n_local = base + (1 if rank < extra else 0)

```

```

# — Seed independen per proses (SeedSequence menjamin bebas korelasi) —
seed_seq    = np.random.SeedSequence(MASTER_SEED)
child_seeds = seed_seq.spawn(size)
local_seed  = int(child_seeds[rank].generate_state(1)[0])

# — Timer mulai (barrier memastikan semua proses mulai bersamaan) —
comm.Barrier()

t_start = MPI.Wtime()    # waktu wall-clock MPI (lebih presisi dari
                        time.time)

# — Komputasi lokal —————
local_results = simulate_local(n_local, local_seed)

# — Kumpulkan hasil di rank 0 (Gather) —————
all_results = comm.gather(local_results, root=0)

comm.Barrier()

t_end = MPI.Wtime()

# — Hanya rank 0 yang mencetak & memplot —————
if rank == 0:
    total_times = np.concatenate(all_results)    # gabung semua event

    elapsed = t_end - t_start
    print("=" * 60)
    print(f"  Jumlah proses MPI      : {size}")
    print(f"  Jumlah event (N)          : {N_EVENTS:,}".replace(",","."))
    print(f"  Waktu komputasi           : {elapsed:.6f} detik")
    print(f"  Rata-rata t peluruhan     : {total_times.mean():.6e} tahun")

```

```

    print(f"    Median    t peluruhan      : {np.median(total_times):.6e}
tahun")

    print(f"    Std dev t peluruhan      : {total_times.std():.6e} tahun")
    print("=" * 60)

    # Plot distribusi

    matplotlib.use("Agg")      # non-interactive; ganti "TkAgg" jika
ingin pop-up

    fig, ax = plt.subplots(figsize=(9, 5.2), dpi=150)

    ax.hist(
        total_times / 1e9,
        bins=50,
        color="#2d6a9f",
        edgecolor="white",
        linewidth=0.4,
    )

    ax.set_xlabel("Total waktu peluruhan U-238 → Pb-206    (miliar
tahun)", fontsize=11)
    ax.set_ylabel("Jumlah event", fontsize=11)
    ax.set_title(
        f"Distribusi waktu peluruhan    |    N = {N_EVENTS:,}    |    {size}
proses MPI".replace(",", " "),
        fontsize=12,
    )

    ax.grid(axis="y", alpha=0.25)
    fig.tight_layout()

    fname = f"hist_mpi_N{N_EVENTS}_p{size}.png"
    fig.savefig(fname)
    print(f"    Histogram tersimpan      : {fname}")

```

```
if __name__ == "__main__":
    main()
```

A. Analisa Performa

Pada percobaan kali ini, digunakan paralelisme pada percobaan peluruhan Uranium-238 dengan jumlah sebanyak 100.000 partikel. Kemudian, dilakukan paralelisme dengan jumlah prosesor/thread yang berbeda yakni 2,4,6,8,10 dan 12 thread untuk menganalisa skalabilitas pada percobaan kali ini. Selain itu, diukur pula juga efisiensi dari tiap kenaikan jumlah thread. Ukuran waktu paralel dan serial yang digunakan adalah dalam bentuk detik. Didapatkan hasil waktu komputasi sebagai yang terlampir pada tabel 1, serta speedup dan efisiensi pada tabel 2.

Jumlah Prosesor	Waktu Komputasi
Serial	0,0525357
2	0,027617
4	0,019391
6	0,015765
8	0,011716
10	0,010938
12	0,011062

Tabel 1. Waktu Komputasi

Berdasarkan Tabel 1, waktu komputasi menunjukkan tren menurun seiring bertambahnya jumlah proses MPI yang digunakan. Pada penggunaan 2 proses, waktu komputasi tercatat sebesar 0,027617 detik. Ketika jumlah proses ditingkatkan menjadi 4, 6, dan 8 proses, waktu komputasi terus berkurang menjadi masing-masing 0,019391 detik, 0,015765 detik, dan 0,011716 detik. Penurunan ini menunjukkan bahwa pembagian beban kerja ke lebih banyak proses mampu mempercepat penyelesaian simulasi karena setiap proses hanya menangani sebagian data yang lebih kecil.

Penurunan waktu komputasi masih terjadi ketika jumlah proses ditingkatkan menjadi 10 proses, yaitu mencapai nilai minimum sebesar 0,010938 detik. Namun, pada penggunaan 12 proses terjadi sedikit kenaikan waktu komputasi menjadi 0,011062 detik. Meskipun kenaikannya relatif kecil, fenomena ini menunjukkan bahwa penambahan jumlah proses tidak selalu menghasilkan percepatan yang lebih baik. Ketika jumlah proses semakin banyak, biaya komunikasi antarproses, sinkronisasi, serta proses pengumpulan data hasil komputasi (gathering)

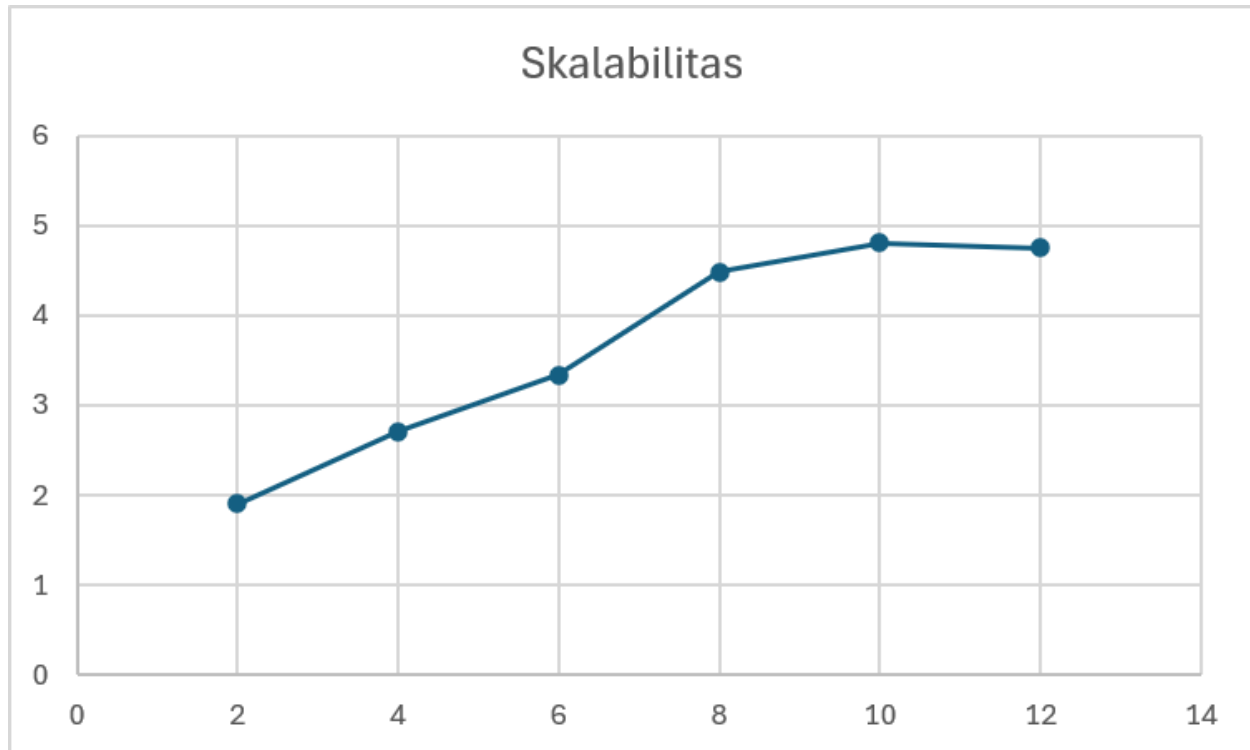
mulai memberikan kontribusi yang signifikan terhadap total waktu eksekusi. Akibatnya, keuntungan yang diperoleh dari pembagian beban kerja menjadi berkurang.

Secara keseluruhan, hasil ini menunjukkan bahwa penggunaan MPI berhasil mengurangi waktu komputasi dibandingkan eksekusi serial. Akan tetapi, terdapat titik optimum pada sekitar 10 proses, di mana penambahan proses setelah titik tersebut tidak lagi memberikan peningkatan performa yang berarti.

Tabel 2. Speedup dan Efisiensi Paralelisme

Jumlah Prosesor	Speedup	Efisiensi
2	1,902	95,11%
4	2,709	67,73%
6	3,332	55,54%
8	4,484	56,05%
10	4,803	48,03%
12	4,749	39,58%

Jika dibentuk sebuah grafik menggunakan data Speedup yang digunakan, maka akan didapatkan pula hubungan speedup dengan jumlah prosesor yang digunakan untuk mengeksekusi sistem paralel maka akan didapatkan pula gambar 1. Sedangkan dengan data efisiensi kita dapat melihat hubungan efisiensi dengan banyaknya prosesor yang digunakan untuk mengeksekusi sistem paralel yang digunakan, maka didapatkan gambar 2.

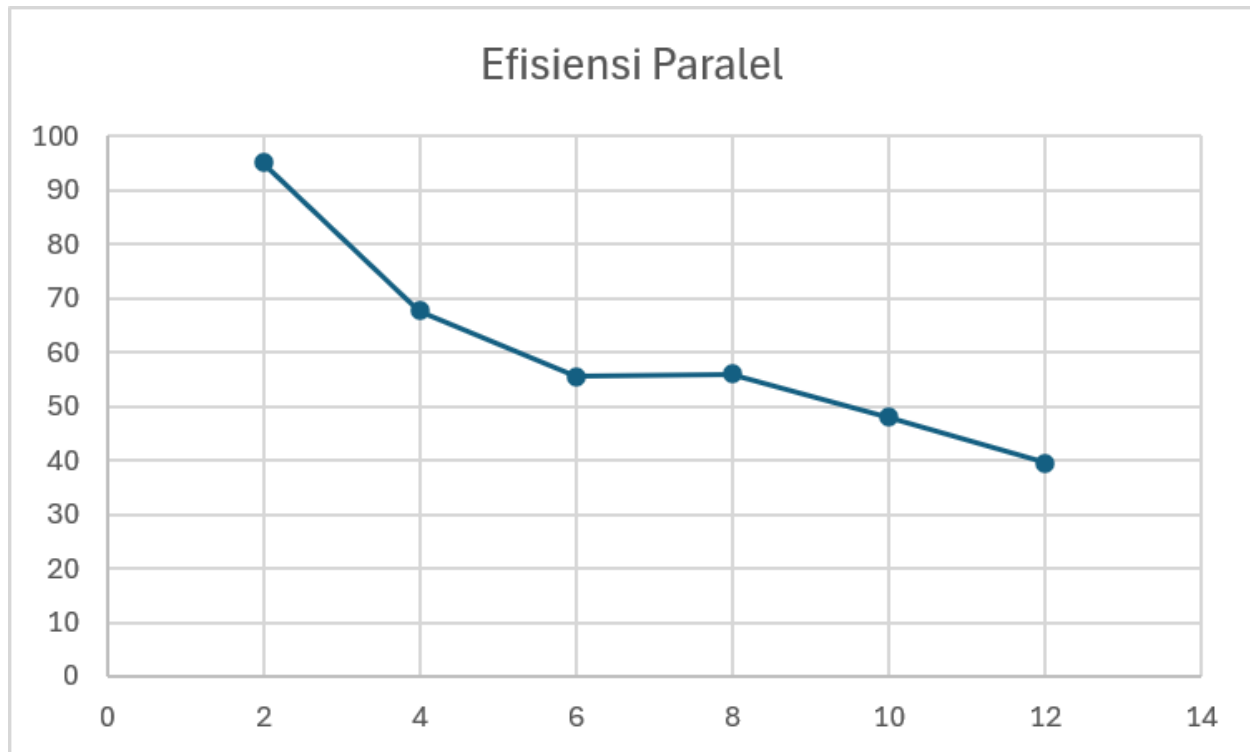


Gambar 1. Grafik Skalabilitas Simulasi Kasus 1

Berdasarkan gambar 1 dan Nilai speedup pada Tabel 2 menunjukkan peningkatan seiring bertambahnya jumlah proses MPI. Pada penggunaan 2 proses diperoleh speedup sebesar 1,902, yang berarti program berjalan hampir dua kali lebih cepat dibandingkan eksekusi serial. Nilai ini meningkat menjadi 2,709 pada 4 proses, 3,332 pada 6 proses, dan 4,484 pada 8 proses. Tren ini menunjukkan bahwa proses paralelisasi berhasil memanfaatkan sumber daya komputasi tambahan untuk mempercepat perhitungan.

Peningkatan speedup mencapai nilai maksimum sebesar 4,803 pada penggunaan 10 proses. Namun, ketika jumlah proses ditingkatkan menjadi 12, speedup justru sedikit menurun menjadi 4,749. Penurunan ini mengindikasikan bahwa overhead komunikasi MPI mulai mengimbangi keuntungan yang diperoleh dari pembagian pekerjaan ke lebih banyak proses. Dengan kata lain, sebagian waktu komputasi digunakan untuk pertukaran data dan sinkronisasi antarprosesor sehingga peningkatan performa tidak lagi sebanding dengan jumlah proses yang ditambahkan.

Jika dibandingkan dengan kondisi ideal, yaitu speedup linear sebesar jumlah proses yang digunakan ($S_{ideal} = P$), nilai speedup yang diperoleh masih berada di bawah nilai teoritis. Sebagai contoh, pada 10 proses speedup ideal seharusnya bernilai 10, sedangkan hasil pengukuran hanya mencapai 4,803. Hal ini menunjukkan adanya bagian program yang tidak dapat diparalelkan secara sempurna serta adanya overhead yang tidak dapat dihindari dalam implementasi MPI. Dilihat pula pada grafik bahwa peningkatan tidak sepenuhnya linear sesuai dengan teoritis.



Gambar 2. Grafik Efisiensi Paralel

Efisiensi paralel menggambarkan tingkat pemanfaatan setiap proses dalam sistem paralel. Berdasarkan Tabel 2, efisiensi tertinggi diperoleh pada penggunaan 2 proses, yaitu sebesar 95,11%. Nilai ini menunjukkan bahwa hampir seluruh sumber daya komputasi yang tersedia dapat dimanfaatkan secara efektif untuk menyelesaikan perhitungan. Seiring bertambahnya jumlah proses, efisiensi mengalami penurunan secara bertahap. Pada 4 proses efisiensi turun menjadi 67,73%, kemudian menjadi 55,54% pada 6 proses. Menariknya, pada 8 proses terjadi sedikit peningkatan menjadi 56,05%, yang kemungkinan disebabkan oleh distribusi beban kerja yang lebih merata pada konfigurasi tersebut. Namun setelah itu efisiensi kembali menurun menjadi 48,03% pada 10 proses dan 39,58% pada 12 proses.

Penurunan efisiensi ini menunjukkan bahwa tidak semua proses dapat dimanfaatkan secara optimal ketika jumlah proses semakin banyak. Semakin besar jumlah proses yang digunakan, semakin besar pula waktu yang diperlukan untuk komunikasi data, sinkronisasi, dan koordinasi antarproses. Akibatnya, sebagian prosesor harus menunggu proses lain menyelesaikan pekerjaannya sehingga tingkat pemanfaatan sumber daya menjadi berkurang.

Hasil pengujian menunjukkan bahwa peningkatan performa mulai mengalami perlambatan pada penggunaan lebih dari 10 proses. Berdasarkan Hukum Amdahl, kondisi ini terjadi karena masih terdapat bagian program yang harus dieksekusi secara serial dan tidak dapat diparalelkan. Dari data yang diperoleh, fraksi serial program diperkirakan berada pada kisaran 4–6%, yang berarti sekitar 94–96% bagian program telah berhasil diparalelkan.

Meskipun persentase paralelisasi tersebut tergolong tinggi, keberadaan fraksi serial yang relatif kecil tetap membatasi speedup maksimum yang dapat dicapai. Selain itu, overhead komunikasi MPI, terutama pada proses pengiriman dan pengumpulan data antarproses, turut berkontribusi terhadap keterbatasan skalabilitas sistem. Oleh karena itu, penambahan jumlah proses di atas titik optimum tidak lagi menghasilkan peningkatan performa yang signifikan.

Secara keseluruhan, implementasi MPI pada simulasi ini menunjukkan skalabilitas yang baik hingga 10 proses, ditandai dengan penurunan waktu komputasi dan peningkatan speedup yang konsisten. Namun, setelah titik tersebut, pengaruh overhead komunikasi dan sinkronisasi menjadi lebih dominan sehingga efisiensi dan speedup mulai mengalami penurunan. Hasil ini sesuai dengan teori komputasi paralel dan Hukum Amdahl yang menyatakan bahwa peningkatan jumlah prosesor tidak dapat meningkatkan performa secara tak terbatas.

B. Analisa Kendala dan Solusi

1. Analisa Kendala

Berdasarkan hasil pengujian performa, beberapa kendala yang memengaruhi skalabilitas program MPI dapat diduga. Kendala pertama adalah overhead komunikasi pada proses pengumpulan data menggunakan fungsi *MPI Gather*. Pada implementasi ini, setiap proses mengirimkan hasil komputasinya ke proses utama (*rank 0*) untuk digabungkan dan dianalisis lebih lanjut. Seiring bertambahnya jumlah proses MPI, jumlah komunikasi yang terjadi juga meningkat sehingga sebagian waktu eksekusi digunakan untuk pertukaran data antarproses. Dampak dari fenomena ini mulai terlihat pada penggunaan 12 proses, di mana waktu komputasi sedikit meningkat dan efisiensi turun menjadi 39,58%.

Kendala kedua dapat terjadi akibat hal yang berkaitan dengan ketidakseimbangan beban kerja (*load imbalance*). Pembagian jumlah event ke setiap proses tidak selalu dapat dilakukan secara merata karena jumlah event simulasi belum tentu habis dibagi oleh jumlah proses yang digunakan. Meskipun algoritma telah mendistribusikan sisa pekerjaan secara otomatis, beberapa proses tetap dapat menerima beban kerja yang sedikit lebih besar dibandingkan proses lainnya. Akibatnya, proses dengan beban terbesar menjadi penentu waktu penyelesaian keseluruhan karena proses lain harus menunggu hingga seluruh proses selesai sebelum tahap penggabungan data dilakukan.

Kendala berikutnya adalah adanya bagian program yang masih berjalan secara serial pada *rank 0*. Setelah seluruh proses selesai melakukan simulasi, *rank 0* bertugas menerima hasil dari semua proses, menggabungkan data menggunakan fungsi `numpy.concatenate()`, serta melakukan visualisasi hasil dalam bentuk histogram. Tahapan ini tidak dapat diparalelkan sehingga menjadi salah satu faktor yang membatasi peningkatan performa ketika jumlah proses terus ditambah.

Selain faktor perangkat lunak, hasil pengujian juga menunjukkan adanya indikasi keterbatasan skalabilitas perangkat keras. Hal ini terlihat dari nilai speedup yang mencapai maksimum pada 10 proses sebesar 4,803 kemudian sedikit menurun menjadi

4,749 pada 12 proses. Kondisi tersebut menunjukkan bahwa penambahan proses tidak lagi menghasilkan peningkatan performa yang signifikan. Fenomena ini dapat dipengaruhi oleh keterbatasan jumlah inti prosesor yang tersedia, bandwidth memori, maupun sumber daya sistem lainnya yang harus dibagi oleh seluruh proses MPI.

2. Solusi yang disarankan

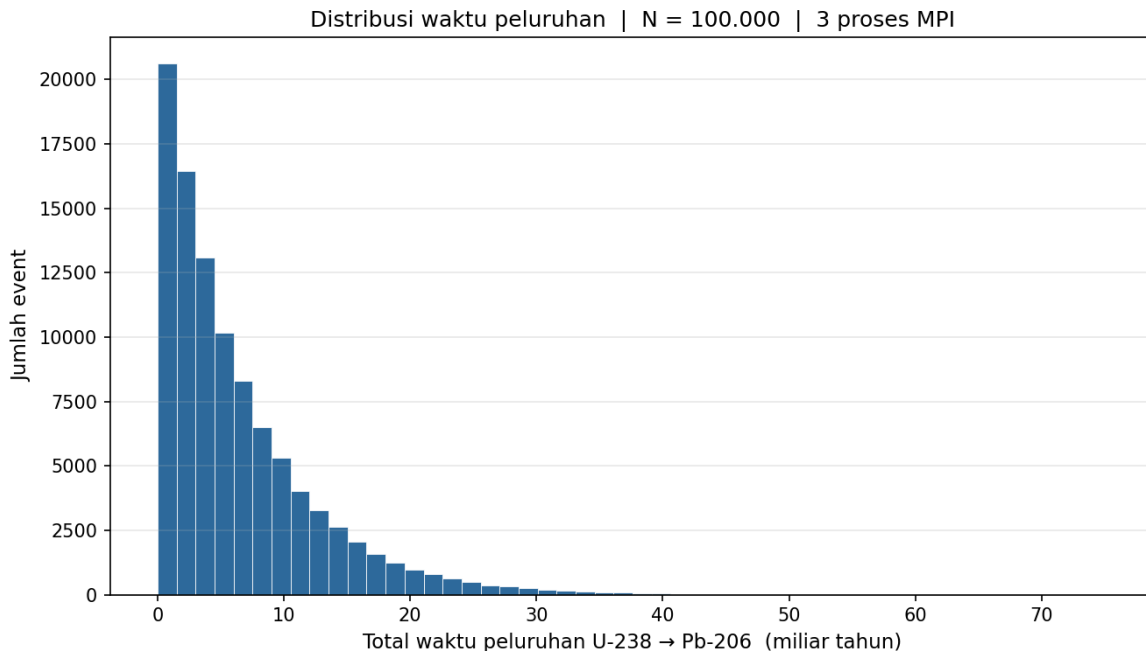
Berdasarkan kendala yang ditemukan, terdapat beberapa pendekatan yang dapat dilakukan untuk meningkatkan performa program pada penelitian selanjutnya. Untuk mengurangi overhead komunikasi MPI, proses pengumpulan data dapat diganti dengan operasi kolektif seperti MPI Reduce apabila hanya diperlukan nilai statistik tertentu. Dengan pendekatan ini, volume data yang ditransfer antarproses dapat dikurangi sehingga komunikasi menjadi lebih efisien.

Masalah ketidakseimbangan beban kerja dapat diminimalkan melalui penerapan dynamic load balancing. Pada metode ini, pekerjaan tidak dibagikan secara tetap di awal eksekusi, melainkan diberikan secara dinamis kepada proses yang telah menyelesaikan tugas sebelumnya. Dengan demikian, seluruh proses dapat dimanfaatkan secara lebih merata dan waktu tunggu dapat dikurangi.

Untuk mengatasi bottleneck pada rank 0, proses pengolahan hasil dapat didistribusikan ke beberapa proses atau memanfaatkan teknik parallel I/O. Pendekatan ini dapat mengurangi beban kerja yang terpusat pada satu proses sehingga bagian serial program menjadi lebih kecil.

Selain itu, apabila ukuran simulasi diperbesar secara signifikan, penggunaan pendekatan komputasi hibrida seperti MPI dan OpenMP atau pemanfaatan akselerator GPU menggunakan CUDA maupun OpenCL dapat dipertimbangkan. Metode tersebut berpotensi memberikan peningkatan performa yang lebih tinggi dibandingkan hanya menambah jumlah proses MPI pada satu sistem komputasi.

C. Hasil Visualisasi Paralel



Gambar 3. Histrogram Distrsbusi Waktu Peluruhan U-238

Pada percobaan kali ini, dihasilkan visualisasi dalam bentuk histrogram distribusi waktu peluruhan Uranium-238. Histogram yang dihasilkan dari simulasi menunjukkan distribusi total waktu peluruhan rantai radioaktif U-238 hingga mencapai Pb-206 stabil. Berdasarkan bentuk histrogram, frekuensi kejadian paling banyak berada pada rentang waktu peluruhan yang relatif lebih pendek, kemudian secara bertahap menurun pada waktu yang lebih besar. Pola ini menghasilkan distribusi yang tidak simetris dan memiliki ekor yang memanjang ke arah kanan (*right-skewed distribution*).

Bentuk distribusi tersebut sesuai dengan karakteristik proses peluruhan radioaktif yang bersifat acak (*random process*). Sebagian besar atom mengalami peluruhan dalam rentang waktu yang tidak terlalu jauh dari nilai rata-ratanya, sedangkan sebagian kecil atom dapat bertahan jauh lebih lama sehingga menghasilkan ekor distribusi yang panjang. Akibatnya, frekuensi kejadian pada waktu peluruhan yang sangat besar menjadi semakin kecil namun tetap muncul pada histogram.

Selain itu, tidak terlihat adanya lebih dari satu puncak (*multimodal distribution*) pada histogram. Hal ini menunjukkan bahwa waktu peluruhan total rantai U-238 → Pb-206 didominasi oleh satu mekanisme utama, yaitu peluruhan isotop U-238 yang memiliki waktu paruh paling panjang dibandingkan seluruh isotop turunannya. Dengan demikian, kontribusi isotop perantara terhadap bentuk distribusi total relatif kecil.

Berdasarkan bentuk histrogram yang diperoleh, simulasi berhasil mereproduksi karakteristik utama peluruhan radioaktif yang diprediksi teori. Distribusi menunjukkan kemencengan positif (*positive skewness*), puncak histogram berada pada sisi kiri, serta terdapat

ekor distribusi yang memanjang ke arah waktu peluruhan yang lebih besar. Karakteristik ini konsisten dengan distribusi eksponensial yang menjadi dasar teori peluruhan inti radioaktif.

Hasil tersebut juga sejalan dengan metode simulasi Monte Carlo yang digunakan dalam berbagai perangkat lunak fisika nuklir modern seperti MCNP dan Geant4, di mana waktu peluruhan disampling menggunakan distribusi eksponensial berdasarkan nilai konstanta peluruhan masing-masing isotop. Kesamaan karakteristik distribusi antara simulasi ini dan referensi ilmiah menunjukkan bahwa algoritma yang digunakan telah mampu merepresentasikan proses peluruhan radioaktif secara fisik dengan baik.